LABORATORY OBSERVATION

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	4
Student Name:	A. Bhanu Prasad
Roll No.:	A24126552068
Date:	30-08-2026

---

1. TITLE

Dynamic To-Do List Application with Interactive DOM Manipulation

2. AIM

To develop a dynamic interactive To-Do List application supporting task creation, task completion toggling, task deletion, and empty state management using JavaScript DOM manipulation.

3. OBJECTIVE

- Capture user input text and append new task items dynamically.
- Implement class list toggling (classList.toggle) to strikethrough completed tasks.
- Handle node removal (li.remove()) for deleting tasks.
- Maintain dynamic task counter and empty list message visibility.

4. THEORY

Dynamic DOM manipulation allows web applications to update content without reloading the page. Methods like document.createElement(), element.remove(), classList.toggle(), and parent.appendChild() enable real-time UI modifications based on user interactions.

5. ALGORITHM / PROCEDURE

1. Build container with task input text field, Add Task button, unordered list (<ul>), and empty message paragraph.
2. Implement updateTaskCount() and checkEmpty() functions to manage state.
3. Attach click event listener to Add Task button.
4. Read and trim input text; alert if input is empty.
5. Create <li> element with task text span, complete toggle, and delete button.
6. Attach click listener to toggle 'completed' CSS class.
7. Attach click listener to delete button to remove list item and update count.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>My To-Do List</title>

    <link rel="stylesheet" href="style.css">
</head>

<body>

    <div class="todo-container">

        <div class="title">
            <h1>My To-Do List</h1>
            <p>Organize your tasks and stay productive</p>
        </div>

        <div class="input-section">

            <input
                type="text"
                id="taskInput"
                placeholder="Enter a task...">

            <button id="addTaskBtn">
                Add Task
            </button>
        </div>

        <p id="emptyMessage">
            No tasks available.
        </p>

        <ul id="taskList"></ul>

        <div class="task-counter">
            Total Tasks: <span id="taskCount">0</span>
        </div>

    </div>

    <script src="script.js"></script>

</body>

</html>
```

style.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
body {
    font-family: Arial, sans-serif;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    background: #f1f5f9;
}
.todo-container {
    width: 90%;
    max-width: 550px;
    background: white;
    padding: 30px;
    border-radius: 15px;
    box-shadow: 0 5px 20px #ccc;
}
h1 {
    text-align: center;
    color: #1e293b;
    margin-bottom: 20px;
}
.input-section {
    display: flex;
    gap: 10px;
    margin-top: 15px;
    margin-bottom: 20px;
}
#taskInput {
    flex: 1;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 7px;
    font-size: 15px;
}
button {
    border: none;
    padding: 10px 15px;
    border-radius: 7px;
    color: white;
    cursor: pointer;
}
#addTaskBtn {
    background: #2563eb;
}
#emptyMessage {
    text-align: center;
    color: #888;
    padding: 15px;
}
#taskList {
    list-style: none;
}
.task-item {
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 12px;
    margin: 10px 0;
```

```
    background: #f8fafc;
    border-radius: 7px;
}
.task-text {
    flex: 1;
}
.completed .task-text {
    text-decoration: line-through;
    color: #888;
}
.complete-btn {
    background: #16a34a;
}
.delete-btn {
    background: #dc2626;
}
.task-counter {
    text-align: right;
    color: #777;
    font-size: 14px;
    margin-top: 15px;
}
```

script.js

```
// Select elements from the DOM
const taskInput = document.getElementById("taskInput");
const addTaskButton = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");
const taskCount = document.getElementById("taskCount");

// Button event
addTaskButton.addEventListener("click", function() {
    const taskText = taskInput.value.trim();

    // Check whether the task field is empty
    if (taskText === "") {
        alert("Please enter a task.");
        return;
    }

    // Create task elements dynamically
    const taskItem = document.createElement("li");
    taskItem.classList.add("task-item");

    const taskTextElement = document.createElement("span");
    taskTextElement.classList.add("task-text");
    taskTextElement.textContent = taskText;

    const completeButton = document.createElement("button");
    completeButton.classList.add("complete-btn");
    completeButton.textContent = "Complete";

    const deleteButton = document.createElement("button");
    deleteButton.classList.add("delete-btn");
    deleteButton.textContent = "Delete";

    // Complete button event
    completeButton.addEventListener("click", function() {
        taskItem.classList.toggle("completed");

        if (taskItem.classList.contains("completed")) {
            completeButton.textContent = "Completed";
        } else {
            completeButton.textContent = "Complete";
```

```
        }
    });

    // Delete button event
    deleteButton.addEventListener("click", function() {
        taskItem.remove();
        updateTaskList();
    });
    // Add elements to task item
    taskItem.appendChild(taskTextElement);
    taskItem.appendChild(completeButton);
    taskItem.appendChild(deleteButton);

    // Add task to webpage
    taskList.appendChild(taskItem);

    // Clear input
    taskInput.value = "";

    // Update task information
    updateTaskList();
});

// Update task list information
function updateTaskList() {
    const totalTasks = taskList.children.length;
    taskCount.textContent = totalTasks;

    if (totalTasks === 0) {
        emptyMessage.style.display = "block";
    } else {
        emptyMessage.style.display = "none";
    }
}

// Enter key event
taskInput.addEventListener("keydown", function(event) {
    if (event.key === "Enter") {
        addTaskButton.click();
    }
});

// Initial state
updateTaskList();
```

7. CODE EXPLANATION

Code / Syntax	Explanation
document.createElement('li')	Creates new list item DOM element for task
span.classList.toggle('completed')	Toggles strikethrough styling class for completed tasks
li.remove()	Removes task list item node from DOM when delete clicked
updateTaskCount()	Recalculates active and total task count

8. TEST CASES AND EXECUTION DATA

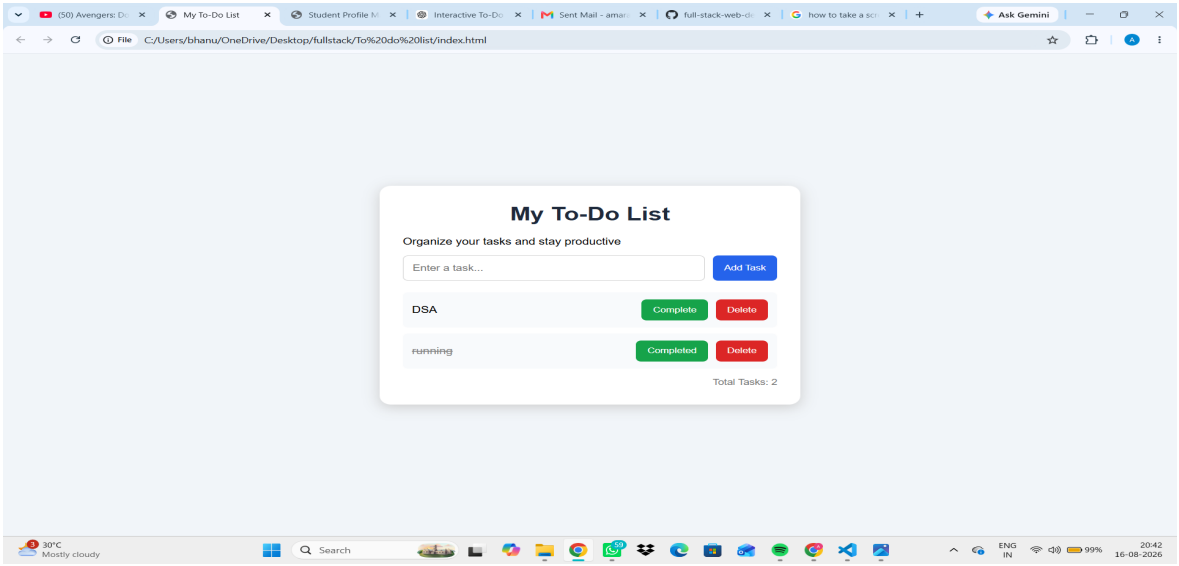
The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Add Task	Enter task name and click Add	Task item appended to list	**Pass**

TC-02	Complete Task	Click complete button on task item	Text gets strikethrough style	**Pass**
TC-03	Delete Task	Click Delete button on item	Task item removed from list	**Pass**
TC-04	Empty List Notice	Delete all tasks from list	Empty state notice displayed	**Pass**

9. OUTPUT

Output 1: Execution Screenshot



10. RESULT

Thus, the experiment for Observation Task 3 (Dynamic To-Do List Application with Interactive DOM Manipulation) was successfully executed and verified.